

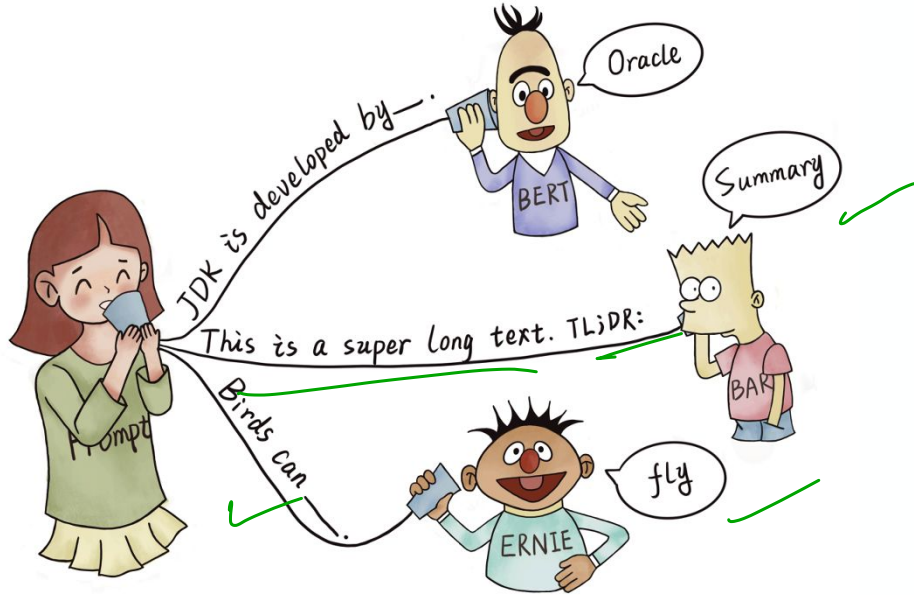
Prompting

CS60216: Safety Fundamentals for Generative AI

Jan 13th, 2026

What is Prompting?

Encouraging a pre-trained model to make particular predictions by providing a textual “prompt” specifying the task to be done.



Basic Prompting

- Append a textual string to the beginning of the sequence and complete

x = **When a dog sees a squirrel, it will usually**

(GPT-2 Small)

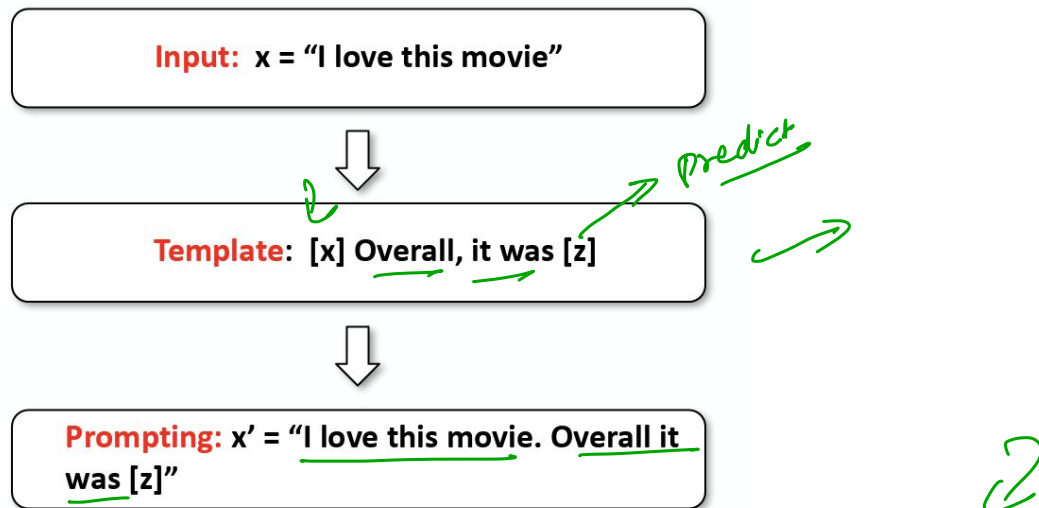
be afraid of anything unusual. As an exception, that's when a squirrel is usually afraid to bite.

(GPT-2 XL)

lick the squirrel. It will also touch its nose to the squirrel on the tail and nose if it can.

Prompt Templates

A template where you fill in with an actual input



I love this movie overall it was

Prompt Templates: Examples

Type	Task	Input ([X])	Template	Answer ([Z])
Text CLS	Sentiment	I love this movie.	[X] The movie is [Z].	great fantastic ...
	Topics	He prompted the LM.	[X] The text is about [Z].	sports science ...
	Intention	What is taxi fare to Denver?	[X] The question is about [Z].	quantity city ...
Text-span CLS	Aspect Sentiment	Poor service but good food.	[X] What about service? [Z].	Bad Terrible ...
Text-pair CLS	NLI	[X1]: An old man with ... [X2]: A man walks ...	[X1]? [Z], [X2]	Yes No ...
Tagging	NER	[X1]: Mike went to Paris. [X2]: Paris	[X1] [X2] is a [Z] entity.	organization location ...
Text Generation	Summarization	Las Vegas police ...	[X] TL;DR: [Z]	The victim ... A woman
	Translation	Je vous aime.	French: [X] English: [Z]	I love you. I fancy you. ...

Examples of input, template, and answer for different tasks. In the Type column, “CLS” is an abbreviation for “classification”

Answer Prediction

Given a prompt, predict the answer

Prompting: $x' =$ "I love this movie. Overall it was [z]"



Predicting: $x' =$ "I love this movie. Overall it was fantastic"



Based on the answer, select the actual output

For instance:

- Taking the output as-is
- Formatting the output for easy visualization
- Selecting only parts of the output that you want to use
- Mapping the outputs to other actions

Output Selection

From a longer response, select the information indicative of an answer

Predicting: $x' =$ "I love this movie. Overall it was a movie that was simply fantastic"



Extraction: fantastic

Various methods for extraction

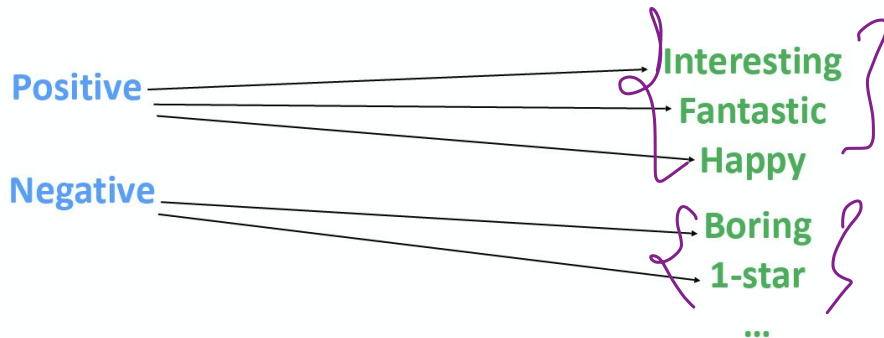
- Classification: identify keywords
- Regression/numerical problems: identify numbers
- Code: pull out code snippets in triple-backticks

Output Mapping

- Given an answer, map it into a class label or continuous value

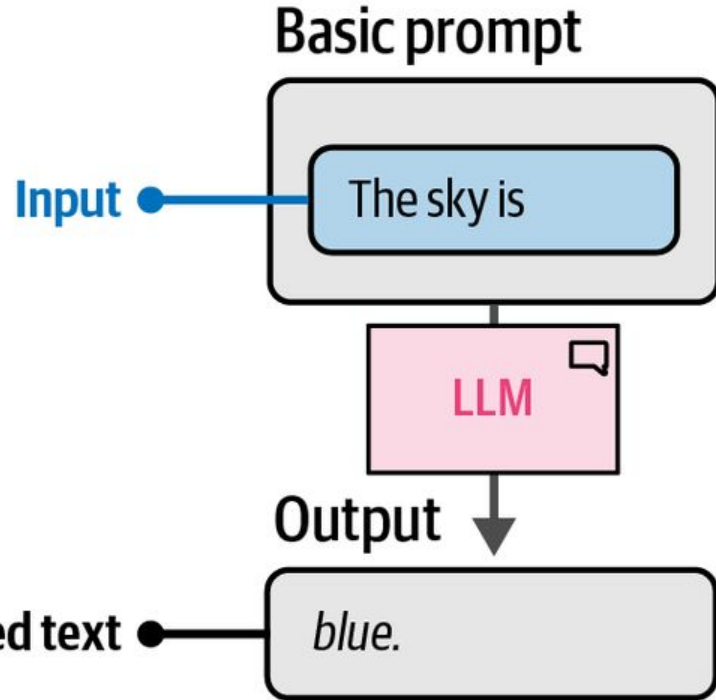


- Often map many extracted words onto a single class



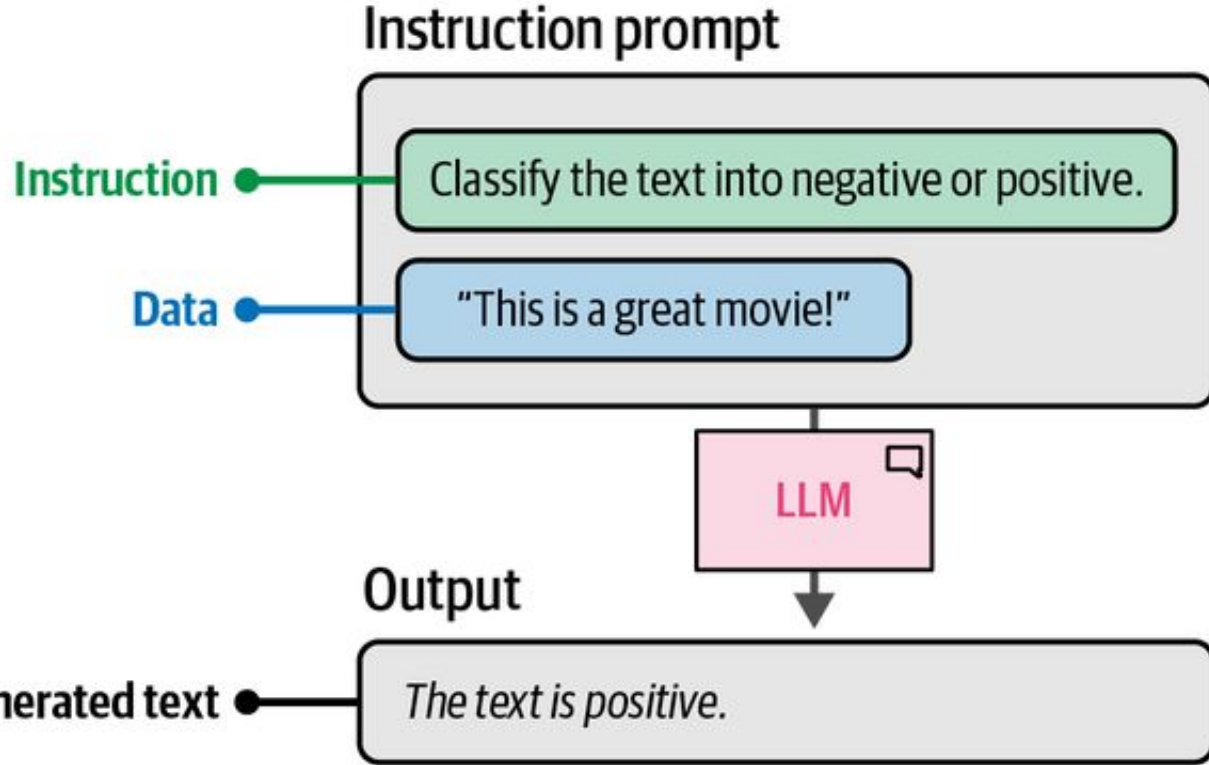
Prompt Variations

Prompt Variations: Basic Prompt



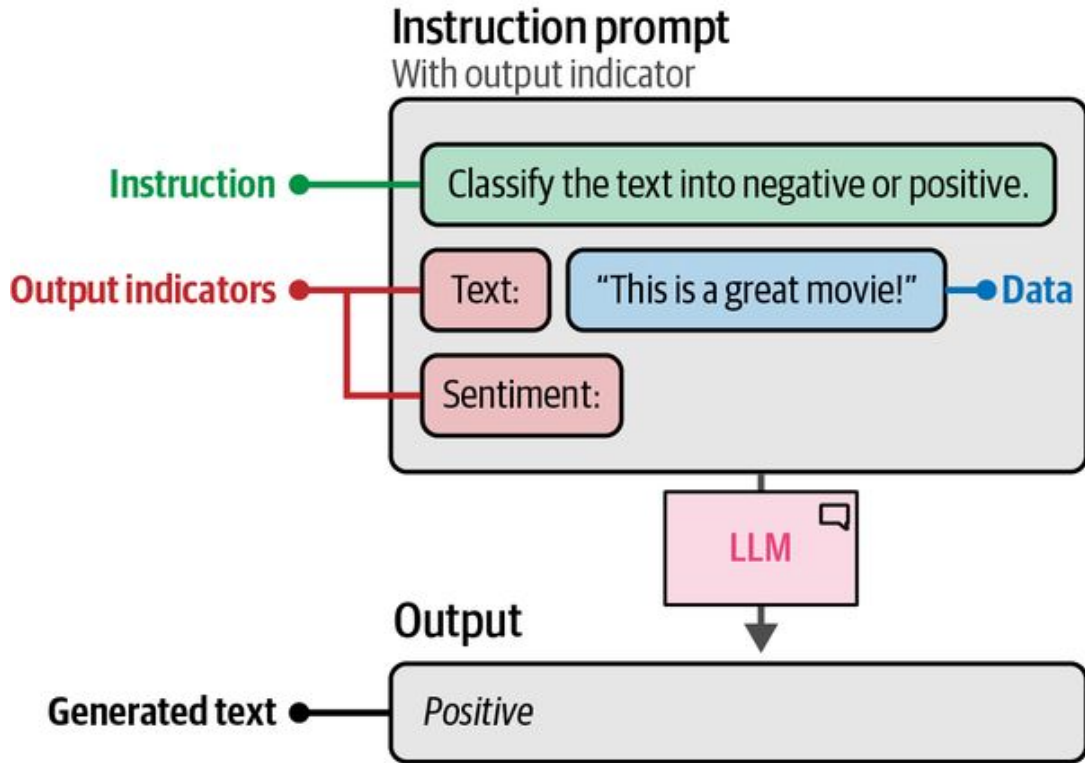
Basic example of a prompt. No instruction is given. So, the LLM will simply try to complete the sentence.

Prompt Variations: Instruction Prompt



Two components of a basic instruction prompt: The instruction itself and the data it refers to

Prompt Variations: Instruction Prompt



Extending the instruction prompt with an output indicator that allows for a specific output.

Chat Prompts Behind the Scenes

- Behind the scenes, messages are converted to token strings

LLaMa

```
Sys. [INST]
      <<SYS>>
      You are an assistant that ...
      <</SYS>>
      [/INST]

User [INST]This movie is great.[/INST]

Asst. Positive.
```

Alpaca

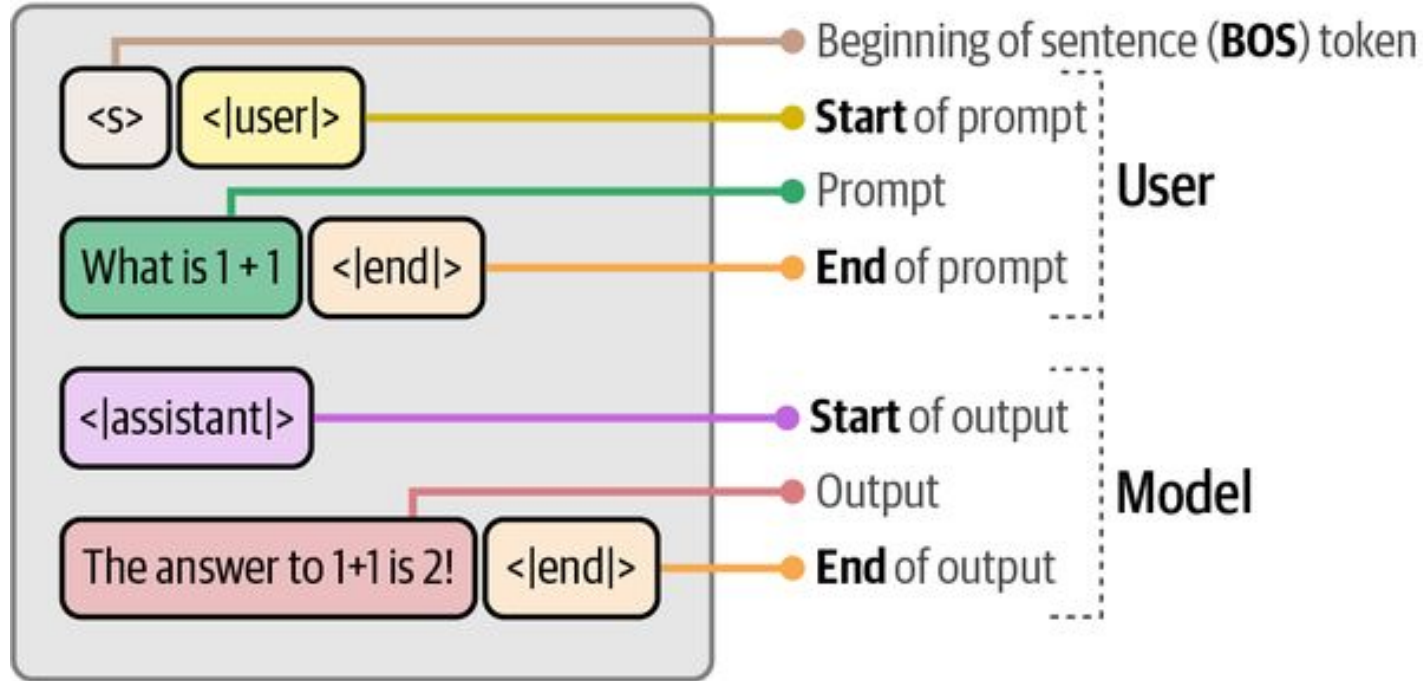
```
Sys. ### Instruction:
      You are an assistant that ...

User ### Instruction:
      This movie is great.

Asst. ### Response:
      Positive.
```

Chat Prompts Behind the Scenes

Phi-3 template



Prompt Engineering Strategies

Describing the task as clearly as possible. *Particularly important when we want the output of the LLM to meet certain expectations.*

Tell me about climate change.

The LLM may generate a response that addresses any aspect of climate change, which may not align with our specific interests. In this case, we can instead use prompts that are specific and detailed.

Prompt Engineering Strategies

Tell me about climate change.



Provide a detailed explanation of the causes and effects of climate change, including the impact on global temperatures, weather patterns, and sea levels. Also, discuss possible solutions and actions being taken to mitigate these effects.

*specific and
detailed prompt*

Prompt Engineering Strategies

Provide a detailed explanation of the causes and effects of climate change, including the impact on global temperatures, weather patterns, and sea levels. Also, discuss possible solutions and actions being taken to mitigate these effects.

Now suppose we intend to explain climate change to a 10-year-old child. We can adjust the above prompt further.

Explain the causes and effects of climate change to a 10-year-old child. Talk about how it affects the weather, sea levels, and temperatures. Also, mention some things people are doing to help. Try to explain in simple terms and do not exceed 500 words.

Asking LLM to follow a reasoning pattern

You are a mathematician. You will follow these detailed reasoning steps when solving math problems.

Step 1: Problem Interpretation.

The mathematician carefully listens to your query and understands the intricate details of the mathematical challenge you have presented.

Step 2: Strategy Formulation.

Drawing upon their extensive knowledge, the mathematician chooses the most effective strategy tailored to the type of math problem, whether it is algebra, calculus, or geometry.

Step 3: Detailed Calculation.

With precision and expertise, the mathematician performs the necessary calculations step by step, adhering to all mathematical principles.

Step 4: Solution Review.

Before providing the final answer, the mathematician meticulously checks the calculations for accuracy and offers a concise explanation or rationale for the solution.

You will be provided with a math problem. Please solve the problem.

{*problem*}

Since solving math problems requires a detailed reasoning process, LLMs would probably make mistakes if they attempted to work out the answer directly. So we can explicitly ask LLMs to follow a given reasoning process before coming to a conclusion

Multiple rounds of interactions

As a first step, we can instruct LLMs to solve the problem directly

You will be provided with a math problem. Please solve the problem.

{*problem*}

As a second step, we prompt LLMs to evaluate the correctness of the answer and, if necessary, rework it to find a better solution.

You will be provided with a math problem, along with a solution. Evaluate the correctness of this solution, and identify any errors if present. Then, work out your own solution.

Problem: {*problem*}

Solution: {*solution*}

More detailed discussion on how to improve prompting through more reasoning later

A complex prompt with many components

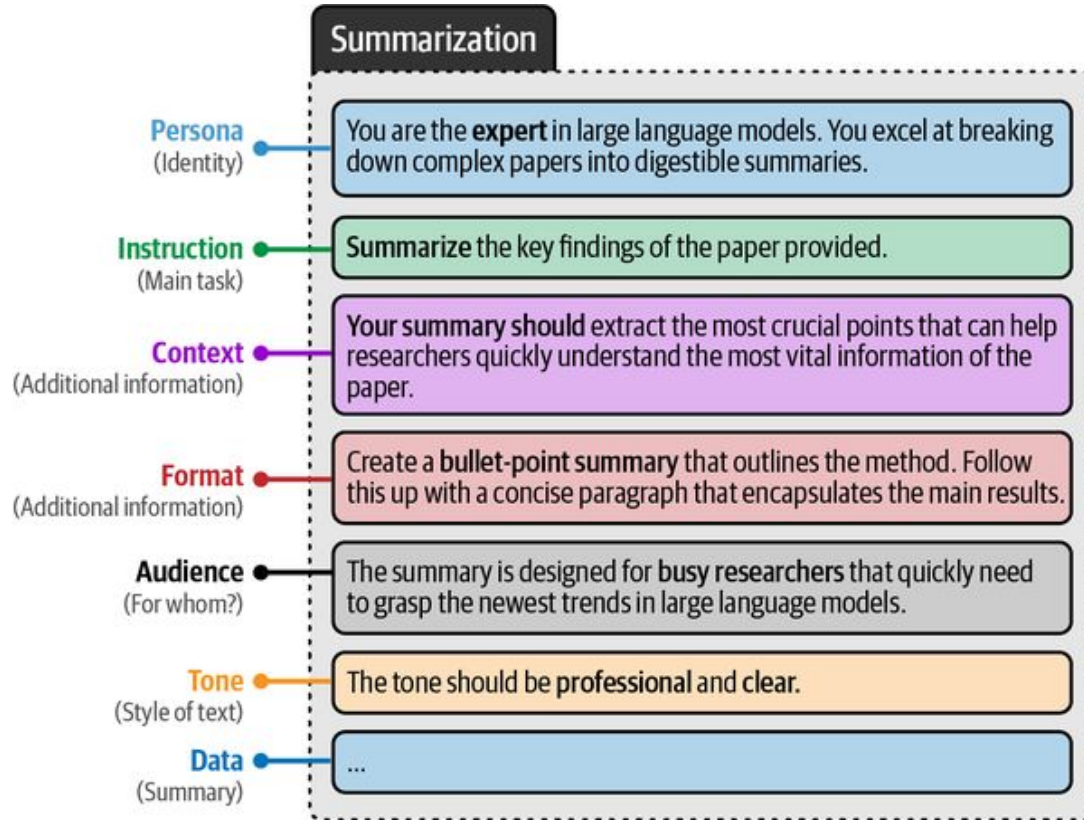


Image Source: Hands-on Large Language Models

Zero-shot vs Few-shot Prompts

Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: I think the food was okay.

Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**.

Text: I think the food was great!

Sentiment: **Positive**.

Text: I think the food was horrible...

Sentiment: **Negative**.

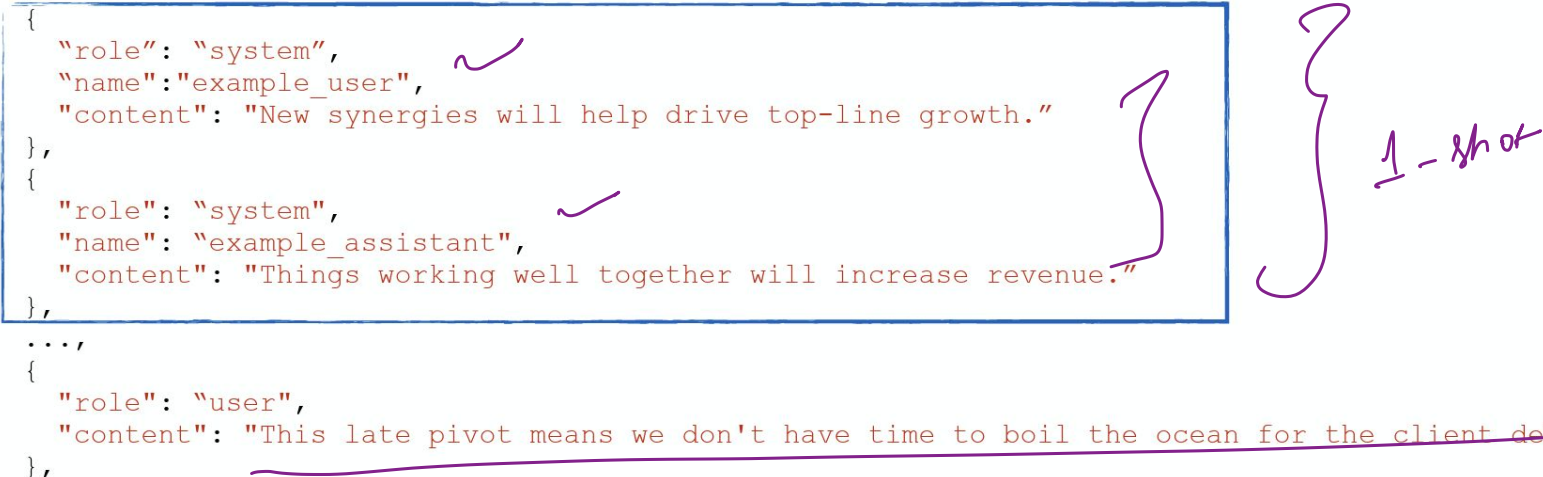
Text: I think the food was okay.

Sentiment:

Few-shot Prompting w/ Chat Prompts

- For OpenAI models, add "role": "system" and a "name": "example_assistant" etc.

```
messages=[
  {
    "role": "system",
    "content": "You are an assistant that translates corporate jargon into plain English."
  },
  {
    "role": "system",
    "name": "example_user",
    "content": "New synergies will help drive top-line growth."
  },
  {
    "role": "system",
    "name": "example_assistant",
    "content": "Things working well together will increase revenue."
  },
  ...,
  {
    "role": "user",
    "content": "This late pivot means we don't have time to boil the ocean for the client deliverable."
  },
]
```



Few-shot learning

We essentially provide a pattern that maps some inputs to the corresponding outputs.

The LLM attempts to follow this pattern in making predictions, provided that the prompt includes a sufficient number of demonstrations, *although generally small*.

SYSTEM You are a helpful assistant, and are great at grammar correction.

DEMO1 You will be provided with a sentence in English. The task is to output the correct sentence.

Input: There is many reasons to celebrate.

Output: There are many reasons to celebrate.

DEMO2 You will be provided with a sentence in English. The task is to output the correct sentence.

Input: Me and my friend goes to the gym every day.

Output: My friend and I go to the gym every day.

USER You will be provided with a sentence in English. The task is to output the correct sentence.

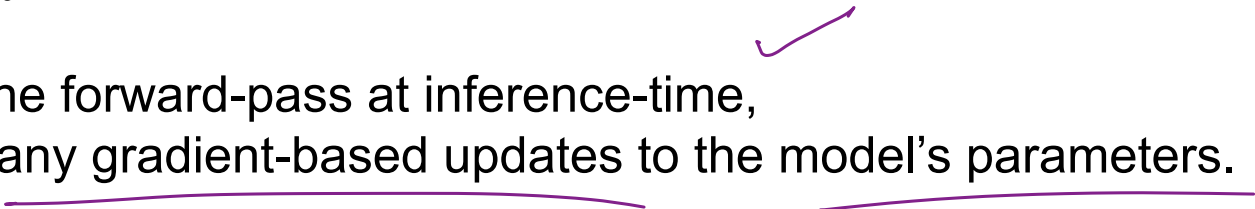
Input: She don't like going to the park.

Output: _____

Why does it work?

Why does in-context learning work?

In-context learning means language models learning to

- do new tasks
 - better predict tokens, or
 - generally reduce their loss
-
- during the forward-pass at inference-time,
 - without any gradient-based updates to the model's parameters.
- 
- A hand-drawn purple checkmark is positioned above the second bullet point of the second list. Below the text of the second bullet point, there are two horizontal purple underlines: one under "without any gradient-based updates" and another under "the model's parameters."

Induction Head Hypothesis

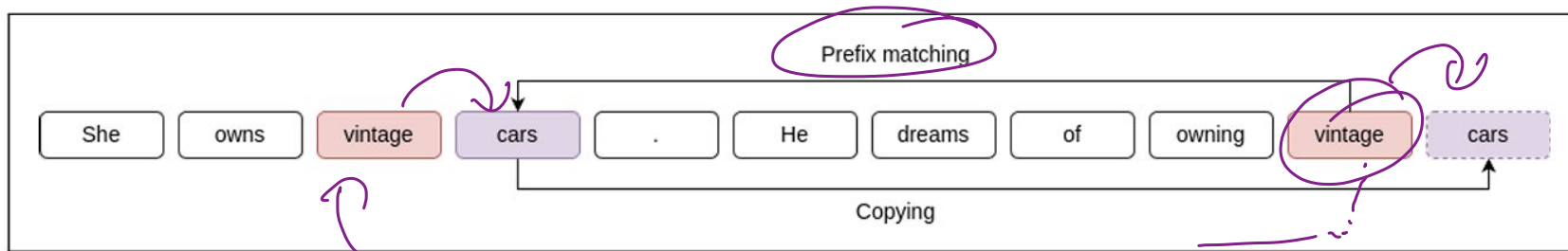


Figure 12.3 An induction head looking at *vintage* uses the *prefix matching* mechanism to find a prior instance of *vintage*, and the *copying* mechanism to predict that *cars* will occur again. Figure from [Crosbie and Shutova \(2022\)](#).

The function of the induction head is to predict repeated sequences.

For example if it sees the pattern *A B ... A* in an input sequence, it predicts that *B* will follow, instantiating the pattern completion rule *A B ... A B*.

Recap: Zero-shot vs Few-shot Prompts

Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: I think the food was okay.

Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**.

Text: I think the food was great!

Sentiment: **Positive**.

Text: I think the food was horrible...

Sentiment: **Negative**.

Text: I think the food was okay.

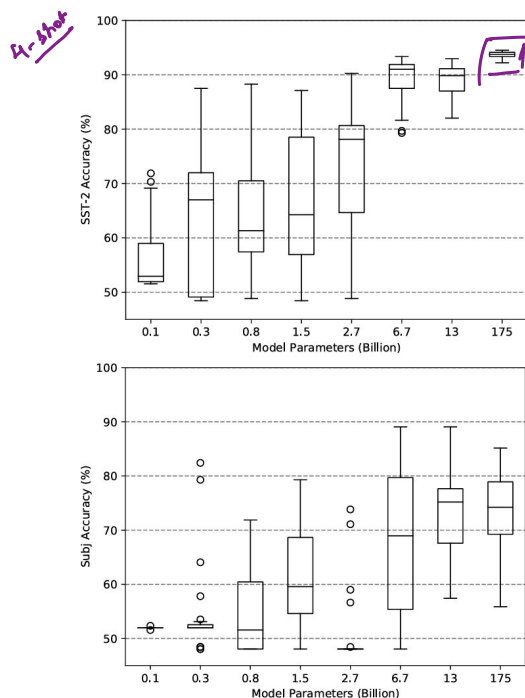
Sentiment:

Interesting questions:

- Which examples to choose?
- Does the order matter?
- Are all labels needed?

....

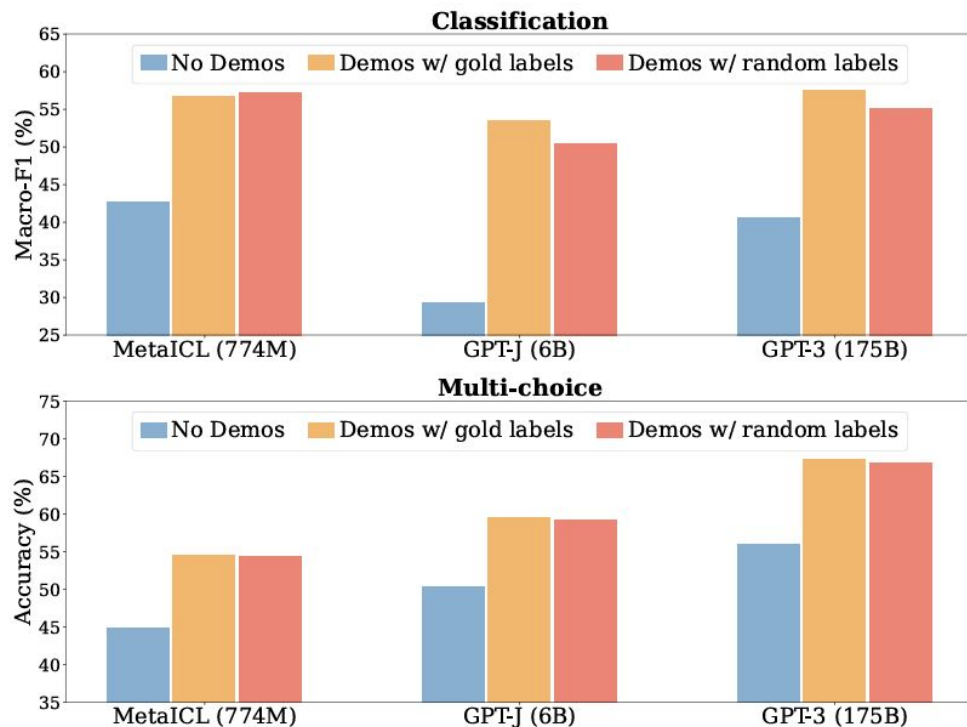
Few-shot prompts: order sensitivity



- Few-shot prompts suffer from order sensitivity, in that for the same prompt **the order in which samples are provided can make the difference** between state-of-the-art and random performance.
- The paper finds that this is **present across tasks, model sizes, prompt templates, samples, and number of training samples.**

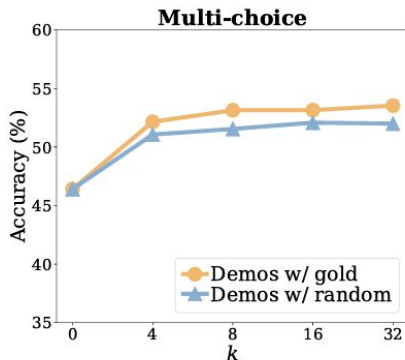
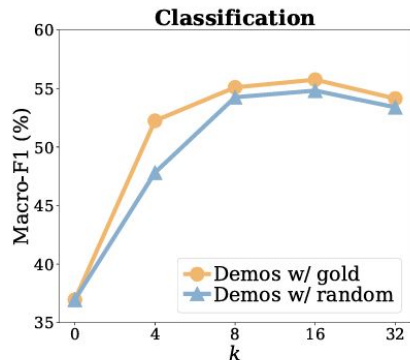
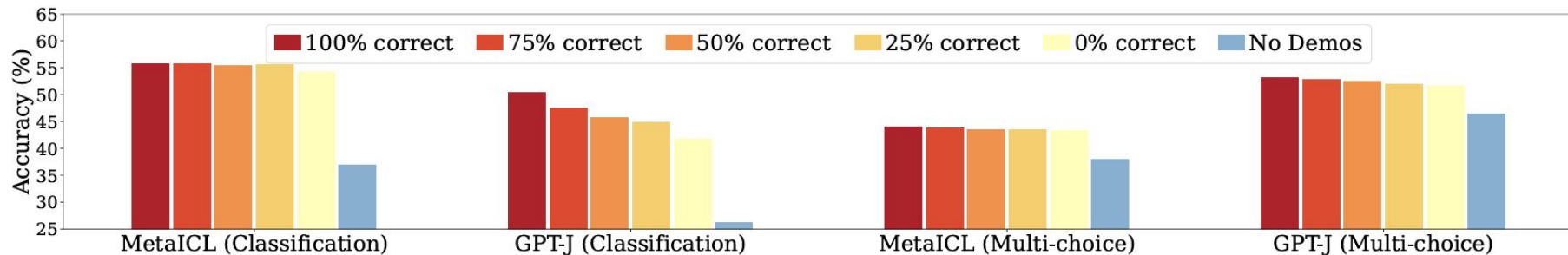
Figure 1: Four-shot performance for 24 different sample orders across different sizes of GPT-family models (GPT-2 and GPT-3) for the SST-2 and Subj datasets.

Some very counterintuitive effects



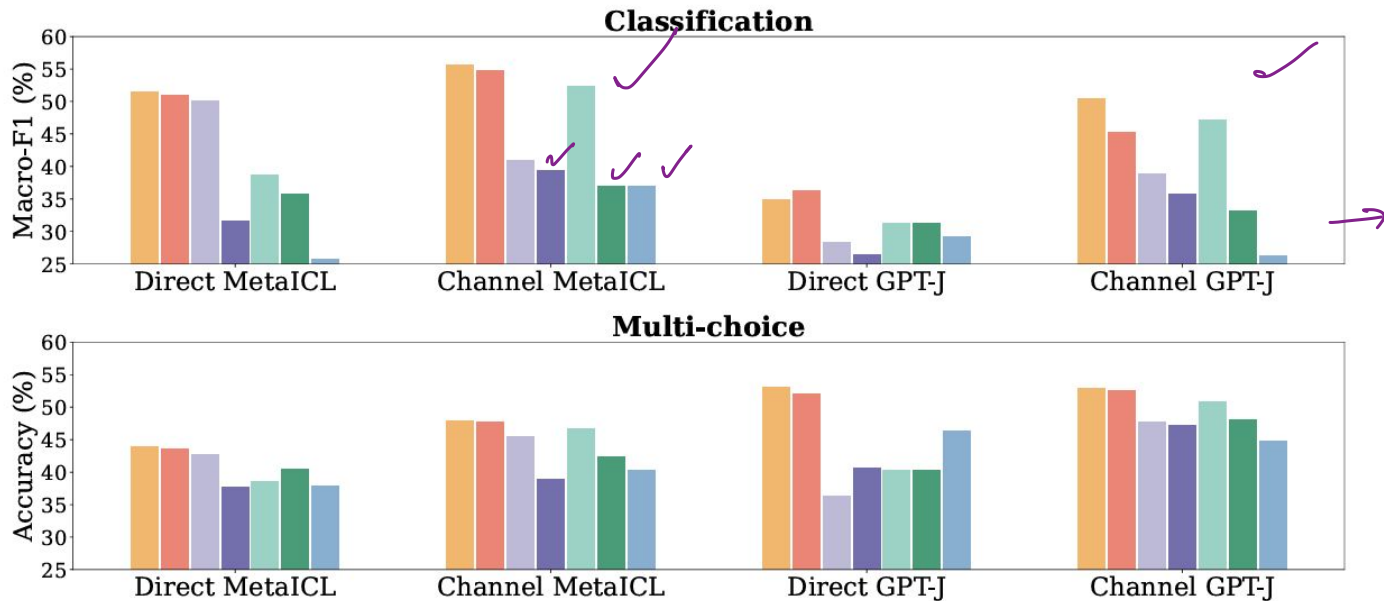
Results in classification (top) and multichoice tasks (bottom), using three LMs with varying size. In-context learning ***performance drops only marginally when labels in the demonstrations are replaced by random labels.***

Some very counterintuitive effects



Ablations on varying numbers of examples in the demonstrations (k)

Why does in-context learning work?



	<i>F</i>	<i>L</i>	<i>I</i>	<i>M</i>
Gold labels	✓	✓	✓	✓
Random labels	✓	✓	✓	✗
OOD + Random labels	✓	✓	✗	✗
Random labels only	✗	✓	✗	✗
Random English words	✓	✗	✓	✗
No labels	✗	✗	✓	✗
No demonstrations	✗	✗	✗	✗

F: Format
L: Label space
I: Input distribution
M: Input-Label Mapping

The format is likely easier to exploit than the input-label mapping

Chain-of-thought (CoT) prompting

Rather than coming to a conclusion directly, the CoT methods instruct LLMs

- to generate reasoning steps or
- to learn from demonstrations of detailed reasoning processes provided in the prompts

Q: Please calculate the average of the numbers 2, 4, and 9.

A: The answer is 6.

Q: Please calculate the average of the numbers 1, 3, 5, and 7.

A: The answer is 4.

Q: Please calculate the average of the numbers 2, 4, and 9.

A: The answer is 7.

Although we have shown a similar question-answer pair, it remains difficult for the LLM to reason out the correct answer

Chain-of-thought (CoT) prompting

In CoT, they may gain more from detailed problem-solving steps that are used to derive the answers.

To do this, we can *incorporate some reasoning steps into the prompt* to obtain a CoT prompt.

Few-shot vs Zero-shot CoT

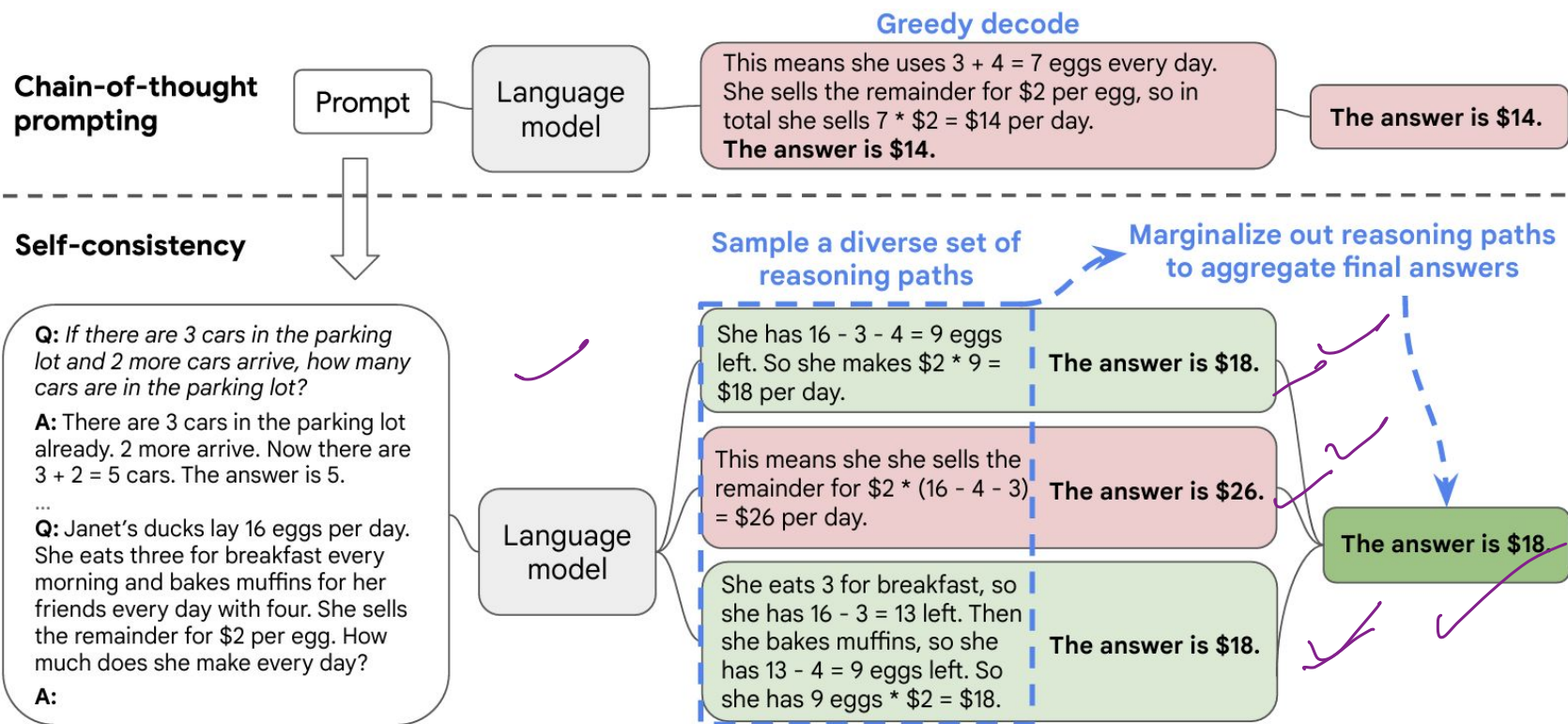
- The method described above requires providing one or more examples of CoT reasoning, typically called the few-shot CoT method.
- By contrast, the zero-shot CoT method does not require such examples. It instead prompts LLMs to reason step-by-step. For example:

Q: Please calculate the average of the numbers 2, 4, and 9. ✓

A: Let's think step-by-step.

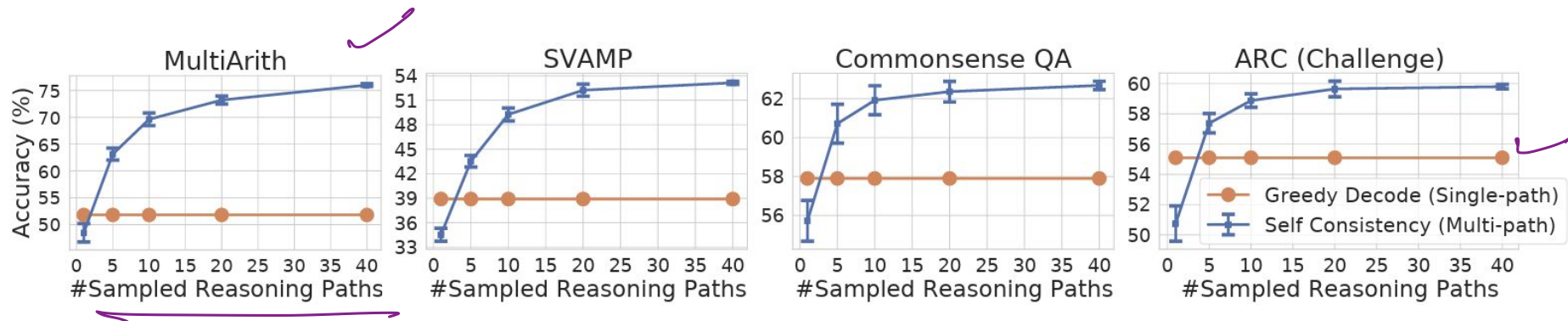
We have three numbers: 2, 4, and 9. Add these numbers together,
 $2 + 4 + 9 = 15$. Determine how many numbers there are, which in this case
is three. The average is calculated by dividing the total sum by the number of
elements. Completing the division gives $15/3 = 5$. So the answer is 5.

Self-consistency with CoT



Self-consistency improves chain of thought reasoning in language models:
<https://arxiv.org/pdf/2203.11171>

Self-consistency with CoT



Self-consistency (blue) significantly improves accuracy over CoT-prompting with greedy decoding (orange) across arithmetic and commonsense reasoning tasks, over LaMDA-137B. Sampling **a higher number of diverse reasoning paths** consistently improves reasoning accuracy.

Self-consistency improves chain of thought reasoning in language models:
<https://arxiv.org/pdf/2203.11171>

Self-Ask

get weather ()

Direct Prompting

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?

Answer: Harry Vaughan Watkins.

Question: Who was president of the U.S. when superconductivity was discovered?

Answer: Franklin D. Roosevelt



Chain of Thought

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?

Answer: Theodor Haecker was 65 years old when he died. Harry Vaughan Watkins was 69 years old when he died.

So the final answer (the name of the person) is: Harry Vaughan Watkins.

Question: Who was president of the U.S. when superconductivity was discovered?

Answer: Superconductivity was discovered in 1911 by Heike Kamerlingh Onnes. Woodrow Wilson was president of the United States from 1913 to 1921. So the final answer (the name of the president) is: Woodrow Wilson.



Self-Ask

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?

Are follow up questions needed here: Yes.

Follow up: How old was Theodor Haecker when he died?

Intermediate answer: Theodor Haecker was 65 years old when he died.

Follow up: How old was Harry Vaughan Watkins when he died?

Intermediate answer: Harry Vaughan Watkins was 69 years old when he died.

So the final answer is: Harry Vaughan Watkins

Question: Who was president of the U.S. when superconductivity was discovered?

Are follow up questions needed here: Yes.

Follow up: When was superconductivity discovered?

Intermediate answer: Superconductivity was discovered in 1911.

Follow up: Who was president of the U.S. in 1911?

Intermediate answer: William Howard Taft.

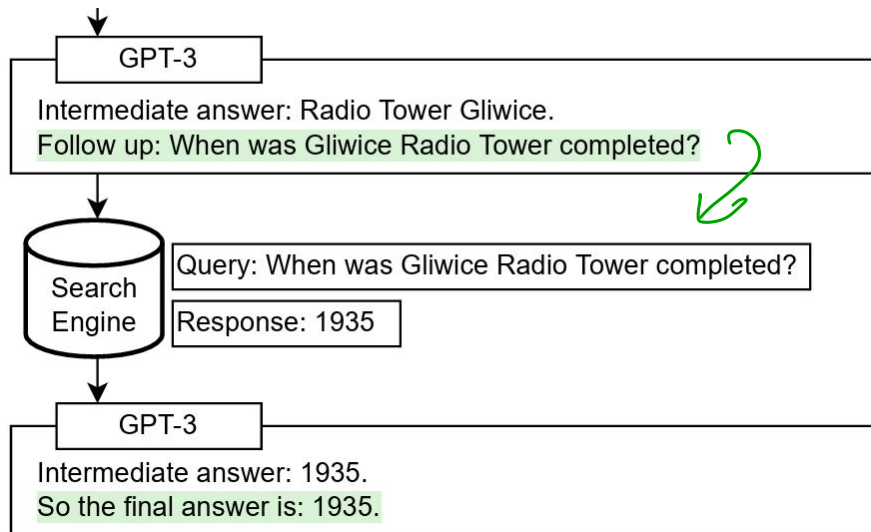
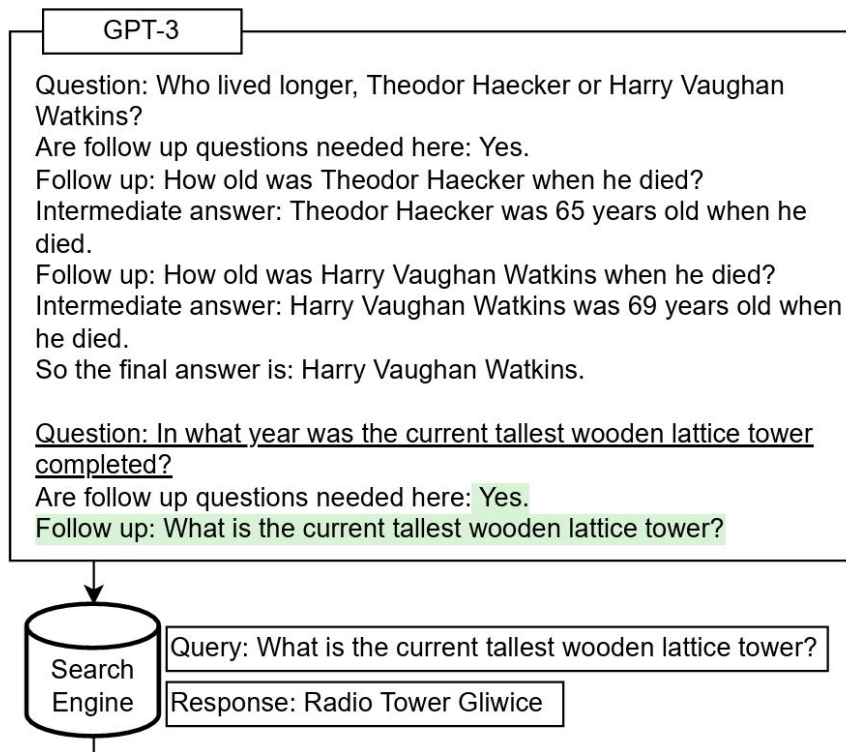
So the final answer is: William Howard Taft.



Text with a white background is the prompt, text with a green background is the LM output, and underlined text is the inference-time question.



Self-Ask + Search Engine



LMs are powerful for text generation tasks.

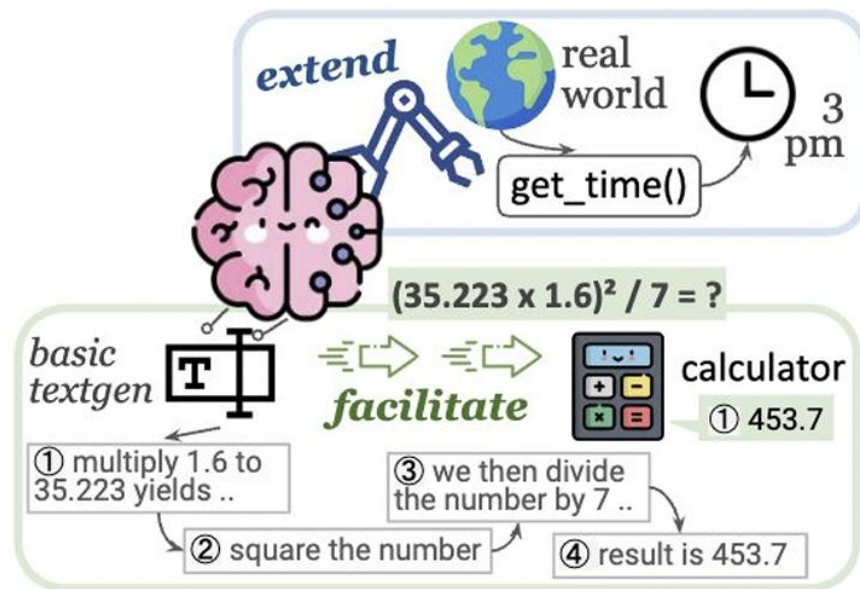
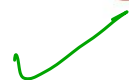
But ...

- Complex reasoning?

Struggle

- Access real-world information?

**Fundamentally
unable**



The Basic Tool Use Paradigm

Tool Use: switching between

- text-generation mode →
- tool-execution mode →

Tool Learning:

- inference-time prompting
- learning by training

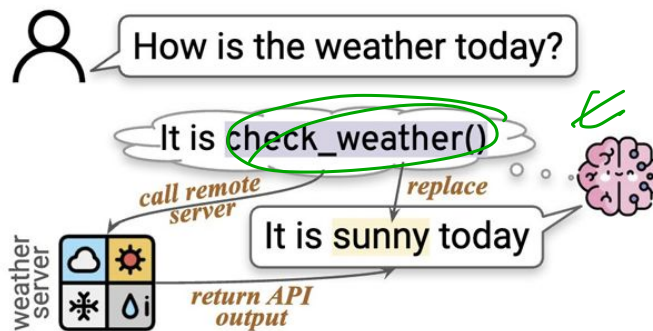


Figure 2: The basic tool use paradigm. LM calls `check_weather` tool by generating text tokens. This call triggers the server to execute the call and return the output `sunny`, using which the LM replaces the API call tokens in the response to the user.

An LM-used tool is a function interface to a computer program that runs external to the LM, where the LM generates the function calls and input arguments in order to use the tool.

Program-of-Thought (PoT) Prompting

Question: In Fibonacci sequence, it follows the rule that each number is equal to the sum of the preceding two numbers. Assuming the first two numbers are 0 and 1, what is the 50th number in Fibonacci sequence?

The first number is 0, the second number is 1, therefore, the third number is $0+1=1$. The fourth number is $1+1=2$. The fifth number is $1+2=3$. The sixth number is $2+3=5$. The seventh number is $3+5=8$. The eighth number is $5+8=13$.
..... (Skip 1000 tokens)
The 50th number is 32,432,268,459.

CoT

```
length_of_fibonacci_sequence = 50
fibonacci_sequence = np.zeros(length_of_)
fibonacci_sequence[0] = 0
fibonacci_sequence[1] = 1
For i in range(3, length_of_fibonacci_sequence):
    fibonacci_sequence[i] = fibonacci_sequence[i-1] +
    fibonacci_sequence[i-2]
ans = fibonacci_sequence[-1]
```

PoT

32,432,268,459



12,586,269,025



- PoT uses language models (mainly Codex) to generate text and programming language statements, and finally an answer.
- In PoT, the computation can be delegated to a program interpreter, thus decoupling complex computation from reasoning and language understanding.

Program-of-Thought (PoT) Prompting

Question: Ketty saves 20000 dollars to the bank. After three years, the sum with compound interest rate is 1000 dollars more than the sum with simple interest rate. What is the interest rate of the bank?

Assuming the interest rate is x . The sum after two years with simple interest rate is $20000 + x * 20000 * 3 = 20000 + 60000x$. The sum after two years with compoud interest rate is $20000 * (1 + x)^3 = 200000 + 60000 * x + 60000x^2 + 20000x^3$. The difference can be written as $60000x^2 + 20000x^3 = 1000$. In order to solve x , we can use the quadratic formula $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, ..., $x = \frac{(-20000 \pm 6160)}{120000}$, $x = -0.051333$.

CoT

```
interest_rate = Symbol('x')
sum_in_two_years_with_simple_interest= 20000 +
interest_rate * 20000 * 3
sum_in_two_years_with_compoud_interest = 20000 * (1 +
interest_rate)**3
# Since compoud interest is 1000 more than simple interest.
ans = solve(sum_after_in_yeras_with_compoud_interest -
sum_after_two_years_in_compoud_interest - 1000,
interest_rate)
```

PoT

-0.051333



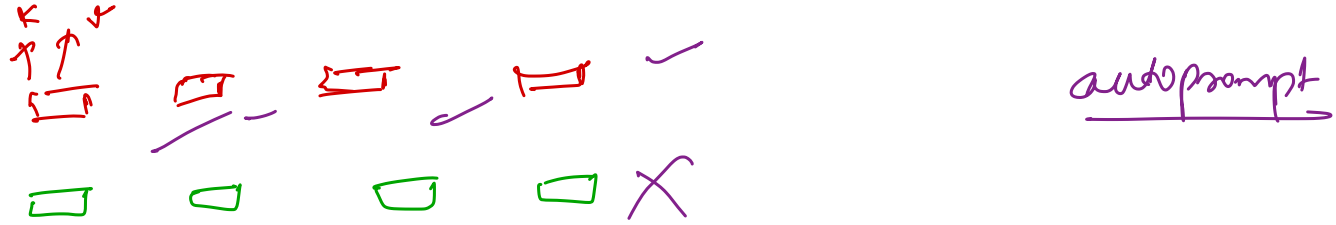
python



SymPy

$x = 0.24814$

- PoT uses language models (mainly Codex) to generate text and programming language statements, and finally an answer.
- In PoT, the computation can be delegated to a program interpreter, thus decoupling complex computation from reasoning and language understanding.

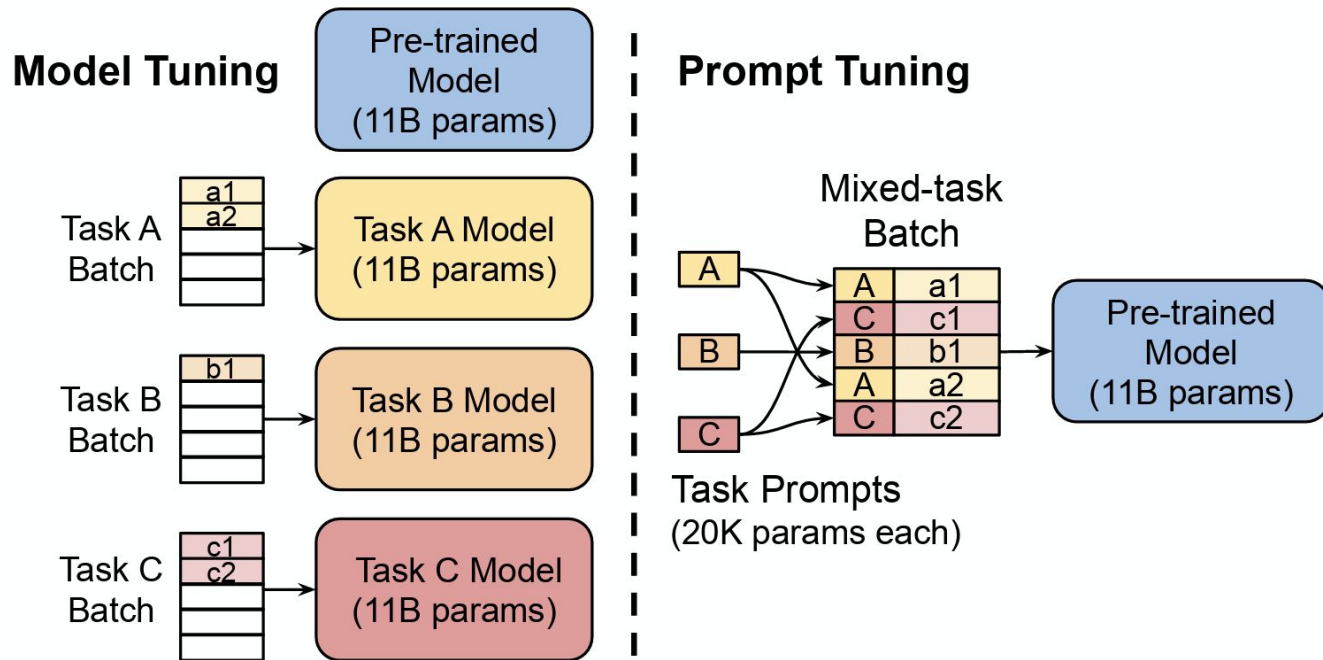


Can we avoid manual prompt engineering?

10 tokens
(80% prompt)

Prompt-tuning

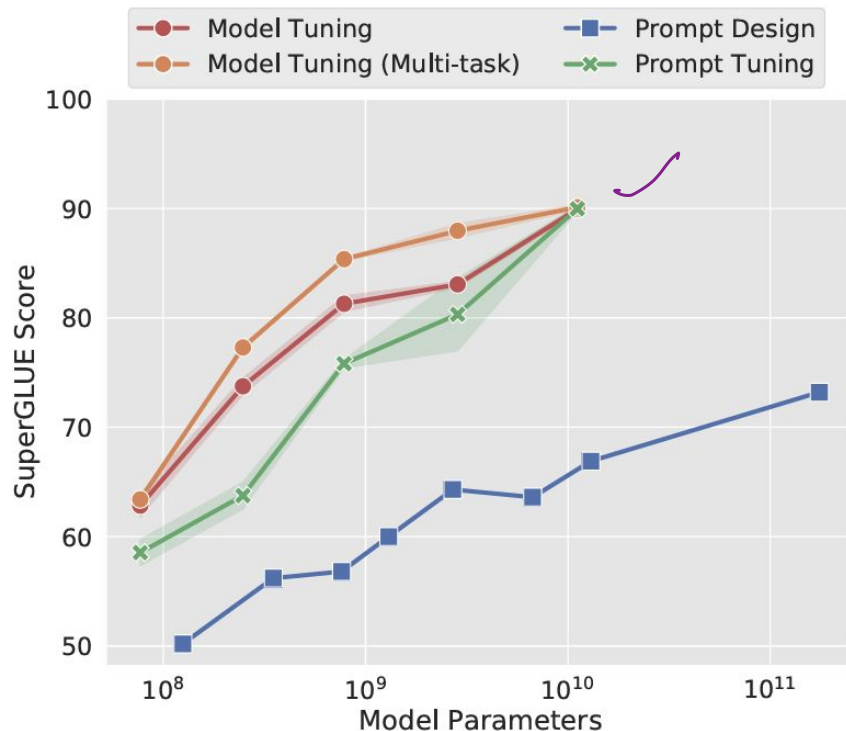
Optimize the embeddings of a prompt, instead of the words.



Prompt-tuning: So, how does it work?

Given a series of n tokens, $\{x_1, x_2, \dots, x_n\}$, the first thing T5 does is embed the tokens, forming a matrix $X_e \in \mathbb{R}^{n \times e}$ where e is the dimension of the embedding space. Our soft-prompts are represented as a parameter $P_e \in \mathbb{R}^{p \times e}$, where p is the length of the prompt. Our prompt is then concatenated to the embedded input forming a single matrix $[P_e; X_e] \in \mathbb{R}^{(p+n) \times e}$ which then flows through the encoder-decoder as normal. Our models are trained to maximize the probability of Y , but only the prompt parameters P_e are updated.

Prompt-tuning: How effective?



Standard **model tuning** of T5 achieves strong performance, but requires storing **separate copies of the model** for each end task. Our prompt tuning of T5 matches the quality of model tuning as size increases, while enabling the reuse of a single frozen model for all tasks. Our approach significantly outperforms fewshot prompt design using GPT-3. We show mean and standard deviation across 3 runs for tuning methods.

*Will talk about more
parameter-efficient methods next week*

The Power of Scale for Parameter-Efficient Prompt Tuning:
<https://aclanthology.org/2021.emnlp-main.243.pdf>

Try this problem

Suppose you want to use T5 model, which has a model dimensionality of 1024 and 24 layers, for the tasks of: (1). text summarization, (2). question answering and (3). text simplification. You can either use full fine-tuning for each task or prompt-tuning (soft-prompts of length 10 for each task). How many extra parameters will you have to store in both of these scenarios at inference time, apart from the T5 parameters?

Inversion Prompting

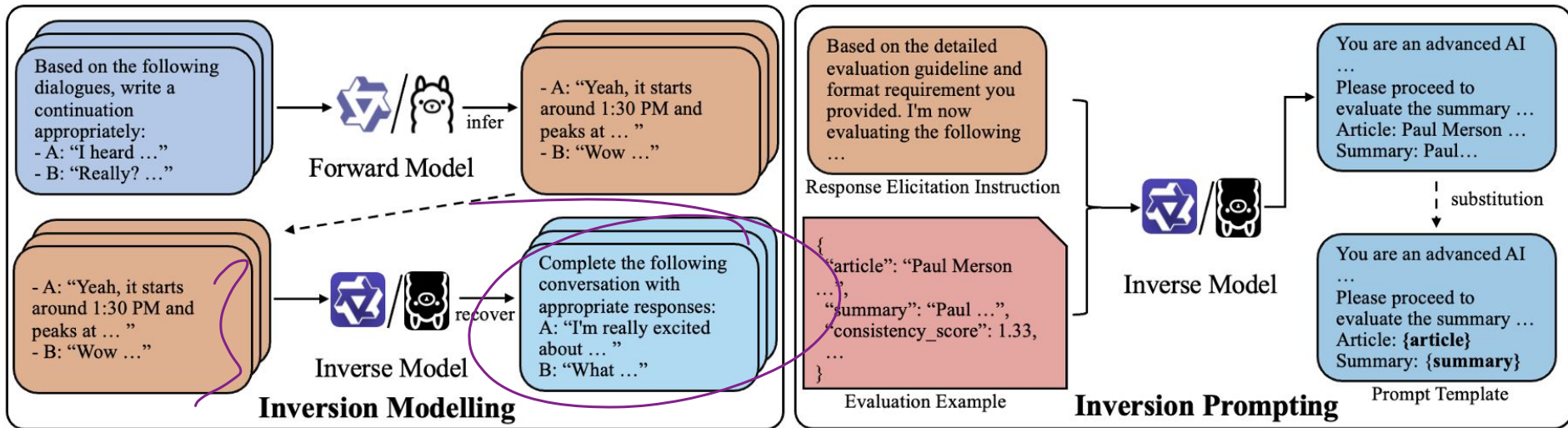


Figure 1: Illustration of the inverse prompt generation process. The bold text in "Prompt Template" indicates substituting the specific example with a generic placeholder.